VHDL 03

Softvér:

Vivado Design Suite - HLx Editions - 2016.2 WebPack

V projekte si zvoľte Artix-7 FPGA čip xc7a35tcpg236-1 (viď. Obr.8 v dokumente vhdl_01.pdf)

Osnova:

- Použitie kľúčového slova **generic** (Kap. 3.5)
- Generujúci príkaz (Kap. 3.9.5)
- Atribúty (Kap. 3.11)
- Agregácia (https://www.edaplayground.com/x/66Ra)
- Práca s komponentmi (Kap. 3.9.6)
- Knižnica UNISIM
- Tvorba testovacieho súboru (Test Bench)
- Simulácia činnosti funkčných jednotiek (FJ)

Použitá literatúra:

[K1] Jiří Pinker, Martin Poupa : Číslicové systémy a jazyk VHDL. BEN - technická literatúra, 2006, ISBN 80-7300-198-5

Praktické ukážky

Definícia generic

```
entity entity_id is
        [generic ( genid : typeid [:=expression]{[; genid: typeid[:=expression]]} );]
        [port ({portid : [in | out | inout | buffer | linkage] typeid [:= expression];});]
end entity_id;
```

Generujúci príkaz:

```
LABEL : gener_statement generate
        [{block declarative items}
        begin]
                { concurrent statements }
        end generate [LABEL];
-- syntax položky gener_statement
        for identifier in discrete_range
-- alebo
        if condition
-- syntax položky discrete_range
        discrete_subtype_indication | simple_expression ( to | downto ) simple_expression
```

Ukážkz generujúceho príkazu

```
wide_and.vhd
if_generate.vhd
ripple_carry_adder.vhd
parity_generator.vhd            -- doplňujúca úloha
```

Atribúty

```
-- syntax použitia atribútov
PREFIX'meno_atribútu[(výraz)]
-- napr.
        signal clk: std_logic;
        …
        if (clk'event and clk = '1') then
                …
        end if;
-- alebo
        variable mystringnumber: string := integer'image(1234);
```

Ukážka použitia atribútu

ripple_carry_adder_tb.vhd

Agregácia

```
-- použitie kľúčového slova others pri definícii/inicializácii vektora
        signal A : unsigned(3 downto 0):= (others => '0');
-- použitie agregácie na pravej strane operátora priradenia
        signal B : std_logic_vector(3 downto 0);
        ...
        B <= (others => '0');
-- použitie agregácie na ľavej strane operátora priradenia
        signal Nibble : std_logic_vector(3 downto 0);
        signal a,b,c,d : std_logic;
        ...
        (a, b, c, d) <= Nibble;
-- Note. https://www.edaplayground.com/x/66Ra
```

Ukážka agregácie

if_generate_tb.vhd

Použitie knižnice UNISIM:

```
library UNISIM;
use UNISIM.VComponents.all;

-- deklarácia entity multiplier

architecture Structural of multiplier is
begin

   -- MULT18X18: 18 x 18 signed asynchronous multiplier

   MULT18X18_inst : MULT18X18 port map (
      P => P,       -- 36-bit multiplier output
      A => A,       -- 18-bit multiplier input
      B => B        -- 18-bit multiplier input
   );
end Structural;
```

Ukážky použitia komponentov z knižnice UNISIM

multiplier.vhd	-- 18 bitová násobička
mult_5x5u_5x5u.vhd	-- Dve násobičky pomocou jedného komponentu

Doplňujúce úlohy

1. Dokončte „test bench“ pre modul multiplier.

   multiplier_tb.vhd

2. Vytvorte „test bench“ pre modul mult_5x5u_5x5u.

   mult_5x5u_5x5u_tb.vhd

3. Dokončte generátor parity. Ak generická konštanta sel = 0, má sa vytvoriť generátor párnej parity. Ak sel = 1, potom sa má vytvoriť generátor nepárnej parity. Pri návrhu použite generujúci príkaz for-generate pre tvorbu štruktúry generátora parity a konštrukciu if-generate na podmienené nastavenie paritného bitu.

   parity_ generator.vhd

4. Vytvorte „test bench“ pre modul parity_ generator.

   parity_ generator_tb.vhd

***Pozn**. Pred prácou s danou FJ nastavte vždy modul ako hlavný modul projektu (Pravý klik na modul a klik na položku Set as Top Module)*